

핵심 역량 (Core Competencies)

클라우드 인프라 설계 및 운영

AWS, GCP 기반 멀티·하이브리드 클라우드 아키텍처 설계 및 운영 경험

- **EKS·GKE 프로덕션 비용 최적화** - 하이브리드 전환으로 20% 절감, GCP 마이그레이션(Fargate → GKE 노드 풀 재배포, MSP 리셀러 할인 확보)으로 50% 절감
- **그린필드 EKS 단독 구축·런칭** - 신규 외부 퍼블리싱 게임의 AWS 프로덕션 환경 — 플랫폼 컴포넌트 100% GitOps 선언적 관리, IRSA / Pod Identity로 정적 키 없는 인증, 2026.07 소프트 런칭 개시
- **IaC 표준화** - 온프레미스·AWS·GCP 전반을 Terraform·Ansible로 코드화 — 신규 컴포넌트 IaC 100%, 변경 이력 100% Git 추적, 동일 파이프라인 30분 내 재구축
- **AI 워크로드 GPU 인프라** - GCP A100 40GB 2장 모델 학습 서버를 Terraform IaC로 구축(너디스타), 온프레미스 GPU 서버에 사내 LLM 추론 서비스 운영(팬텀)

CI/CD 파이프라인 구축 및 GitOps

GitOps 기반 배포 자동화로 개발 생산성 극대화

- **파이프라인 설계·고도화** - GitHub Actions·GitLab CI·Jenkins·ArgoCD 기반 멀티클러스터 CI/CD 파이프라인 설계·배포 자동화
- **배포 생산성·신뢰성** - 배포 시간 67% 단축, GitOps 지속 배포 전환으로 시장 대응 속도 향상
- **카나리 무중단 롤아웃** - 프로덕션 게임 서비스를 Blue-Green → Argo Rollouts Canary로 전환해 승격 시 발생하던 503 레이스 제거
- **CI 튜체인 표준화** - 공용 튜체인 이미지 계층화와 자동 버전 사이클로 CI 빌드 환경 프로비저닝 5분 → 약 10초(약 95% 단축), 도구 버전 일관성 100%

모니터링 및 오피서빌리티 시스템 구축

장애 사전 감지 및 신속한 대응 체계 수립

- **선언적 로깅 인프라 현대화** - ELK → EFK → ECK Operator 기반 Elastic Stack 9.x CRD 선언적 관리 (TLS 자동 회전, 롤링 업그레이드 자동화)
- **메트릭 모니터링 체계** - kube-prometheus-stack 기반 커스텀 알림 규칙·Grafana 대시보드, 멀티 DB exporter(MySQL, Redis, Elasticsearch, PostgreSQL) 통합
- **분산 추적 비용·정확도 최적화** - OpenTelemetry + Tempo로 Metrics ↔ Traces ↔ Logs 3-signal 관측성 확보, 샘플링 지점 분리로 메트릭 정확도와 trace 저장 비용 절감

자동화 및 팀 역량 강화

Python·Bash 운영 자동화 도구 구축과, 고객사 운영팀이 스스로 운영하도록 만드는 교육·자료 표준화

- **운영 튜체인 자동화** - Shell → Python(표준 라이브러리 기반) 마이그레이션, wave 순차 자동 배포와 업그레이드 MR 자동 생성 파이프라인 구축
- **엔지니어링 교육·운영 정착** - DP사 운영팀 대상 Kubernetes 6개월 실습형 교육 설계·운영(수강자 평균 10명·회당 4시간), 자료 표준화로 타 고객사 재사용
- **오픈 엔지니어링 지속 실천** - 공개 API 10여 곳의 금융·경제 데이터를 수집·검증·재배포하는 파이프라인을 평일 하루 2회 자동 실행(**경제 대시보드**), 백엔드 없는 **다이어그램 에디터**도 공개 운영

Kubernetes 플랫폼 운영 고도화 및 오픈소스 기여

네트워크·로깅 스택 전환, 공용 차트 오픈소스 공개, 업스트림 기여

- **Gateway API 마이그레이션** - Ingress-nginx 다수 인스턴스를 NGINX Gateway Fabric 단일 컨트롤플레인으로 마이그레이션 (사고 없이 전체 cutover, wildcard TLS 통합)
- **오픈소스 공용 자산 배포** - 오픈소스 Helm 차트 다수를 비롯해 Kubernetes CRD 컨트롤러, Composite GitHub Actions, Ansible 컬렉션 다수 공개 (ArtifactHub / GitHub Marketplace / Ansible Galaxy 등록)
- **CNCF 업스트림 기여** - apache/airflow · OpenTelemetry · prometheus-community 등의 Helm 차트에는 Gateway API HTTPRoute 지원 기능을, Go 프로젝트에는 버그 수정을 기여

인프라 보안 & IAM

Keycloak Operator SSO와 Vaultwarden으로 사내 인증·시크릿 거버넌스 일원화

- **중앙 인증(SSO/OIDC) 체계** - Keycloak Operator 도입으로 기존 GitLab 계정 기반 단일 로그인 구축, ArgoCD / Harbor / Vaultwarden OIDC 연동 및 검증 완료
- **사내 시크릿 거버넌스** - Vaultwarden을 Kubernetes에 Helm 배포, GitLab SSO·자동 백업·대화형 restore 스크립트로 중앙 관리·복구 체계 구축
- **GitOps 시크릿** - Git에 평문 커밋되던 DB 비밀번호를 External Secrets Operator + AWS Secrets Manager로 전환, Harbor pull secret은 Sealed Secrets로 표준화

주요 경력 (Professional Experience)

팬텀(콘크리트 스튜디오)

2024.10 ~ 현재

DevOps Engineer

게임 개발사의 인프라를 단독으로 책임지는 DevOps 엔지니어로, dev-qa는 온프레미스에, review-prod는 AWS에 두는 하이브리드 아키텍처를 설계·구축·운영해 월 인프라 비용을 20% 절감했습니다.

라이브 게임은 외부 퍼블리셔와 협업해 2차 기술지원·운영을 담당하고, 신규 게임 프로젝트의 개발 생산성 향상을 위해 온프레미스·AWS 인프라를 구축하고 있습니다.

프로젝트 1: AWS-온프레미스 하이브리드 클라우드 아키텍처 구축

기여도: 하이브리드 아키텍처 설계·구축 100% (단독) / 라이브 게임 2차 기술지원 및 운영은 외부 퍼블리셔 운영팀과 협업 2024.10 ~ 현재 (운영 중)

문제

전체 인프라를 AWS 단일 환경에서 운영하며 클라우드 비용 부담이 컸습니다. 트래픽 변동이 적은 비프로덕션 환경까지 클라우드 리소스를 상시 점유하고 있어 비용 최적화와 환경 분리가 필요했습니다.

접근 전략

- 하이브리드 아키텍처 설계** - 워크로드별 비용 및 사용 패턴을 분석해 dev-qa 환경은 온프레미스로 전환하고, 확장성과 고가용성이 필요한 review-prod 환경은 AWS에 유지
- 네트워크 및ファイ프라인 격리** - 복잡한 사이트 간 VPN 구성 없이 애플리케이션 레벨에서 트래픽을 분리하고, 이미지 레지스트리로 이원화 (dev-qa=Harbor, review-prod=ECR)하여 환경별로 독립적으로 빌드·운영
- IaC 기반 운영 표준화** - 온프레미스와 AWS 전반에 Terraform과 Ansible을 도입해 신규 컴포넌트 IaC 100%를 달성하고, 이중 인프라 환경에서도 일관된 형상 관리 체계 수립
- IAM 자격증명 거버넌스** - 서비스·CI 계정만 코드로 관리하고 콘솔 계정은 분리. Terraform state 파일에 장기 자격증명(엑세스 키)이 평문으로 남지 않도록 사용자 키 자동 생성을 배제하고 개별 발급·회전 정책 적용

트리블슈팅

- 클라이언트 버전별 백엔드 분기 라우팅** - EKS 단일 Ingress 환경에서 구버전과 신버전 게임 클라이언트 요청을 서로 다른 백엔드로 분기해야 하는 요구사항 발생.
ALB HTTP 커스텀 헤더 기반의 조건부 라우팅 규칙과 타깃 그룹 매핑을 조합해 추가 인프라 구성 없이 유연한 분기 라우팅 구현
- 롤링 배포 시 L7 커넥션 단절 방지** - ALB 타깃 등록 해제 지연(deregistration delay)과 SIGTERM 타이밍이 어긋나 종료 중인 Pod로 신규 요청이 유입되거나 진행 중인 요청(in-flight)이 끊김. preStop hooks로 헬스체크를 먼저 실패시켜 신규 유입을 차단하고 드레인 지연 동안 잔여 트래픽을 소진, RollingUpdate는 파드 우선 생성(add-before-remove)으로 조정해 롤아웃 중 용량 유지 — 커넥션 단절 0건.
CloudWatch 기반 Grafana에서 ELB 5xx·Target 5xx를 분리해 배포 전후 대조, 5xx 0건·p95 유지 확인

성과

- 워크로드 분석 기반의 하이브리드 재배치로 월 인프라 비용 20% 절감 및 절감분의 신규 프로젝트 재투자에 기여
- 개발·QA와 프로덕션 환경의 물리적 격리를 통해 장애 전파를 원천 차단해 영향 범위(blast radius)를 한정하고 운영 안정성 확보

프로젝트 2: 게임 애플리케이션 CI/CD 파이프라인 구축 및 공용 툴체인 표준화 (App GitOps)

기여도 100% (파이프라인·CI 표준화 전 과정 단독 설계·구축) 2024.12 ~ 현재 (운영 중)

게임 애플리케이션의 수동 빌드·배포를 GitLab CI와 ArgoCD 기반 GitOps 파이프라인으로 자동화하고, 그 위에 사내 repo 전체가 공유하는 CI 툴체인 이미지 계층과 버전 거버넌스를 표준화해 빌드 환경 프로비저닝과 도구 버전 관리를 한 흐름으로 묶었습니다.

2-1. 게임 애플리케이션 배포 CI/CD 파이프라인 (App GitOps, 2024.12 ~ 현재)

문제

개발자가 수동으로 빌드 후 서버에 직접 배포하는 방식이라 배포당 평균 30분이 소요되었고 휴먼 에러로 인한 서비스 장애가 잦았습니다.

접근 전략

- GitOps 기반 지속 배포(CD) 파이프라인 구축** - GitLab CI와 ArgoCD를 연계해 Git Push를 트리거로 빌드 → 테스트 → 배포 전 과정을 자동화하고, Git에 선언된 형상과 클러스터 상태 간 불일치를 감지해 자동 복구(Self-healing)하도록 구성. dev-qa·review-prod 4개 환경의 배포 로직을 단일 파이프라인으로 통합해 관리 일관성 확보
- 최소 권한 기반의 안전한 이미지 빌드 환경 수립** - Docker-in-Docker(DinD) 방식에서 요구되는 호스트 관리자 권한(Privileged 모드) 부여를 배제하기 위해, 도커 데몬 없이 사용자 영역(User-space)에서 컨테이너 이미지를 빌드하는 Kaniko를 도입해 CI 러너의 보안 취약점과 권한 탈취 리스크를 원천 차단

트리블슈팅

- ArgoCD 연쇄 삭제(cascading prune) 방지** - Git 커밋 오류로 하위 Application 전체가 연쇄 삭제되는 사고를 방지하기 위해, 워크로드 Application은 `prune: true` + `selfHeal: true` 로 자동 일치시키고 최상위 app-of-apps 루트·AppProject 계층은 `prune: false` 로 두어 리소스 삭제는

수동 검증을 거치도록 통제.

모든 sync 이벤트는 Slack 알림으로 실시간 추적

- **정적 데이터 배포 경로에서 SSH 키 제거** - CI 배포 계정이 SSH 키를 상시 보유한 채 대상 인스턴스에 직접 접속(scp/rsync)하던 구조를, 산출물을 S3 중계 버킷에 올리고 SSM SendCommand로 인스턴스가 스스로 내려받는 방식으로 교체 — 배포 환경의 정적 SSH 키를 완전히 제거해 자격증명 유출 리스크 차단

성과

- 배포 리드타임 67% 단축 (30분 → 10분) — 빌드·배포·설정 수동 단계를 파이프라인으로 일원화해 휴먼 에러로 인한 배포 장애 원천 제거
- 데이터 배포 소스 페치 시간 약 78% 단축 (약 139초 → 30초) — 최신 커밋과 필요한 파일만 선별해 받는 Git 최적화(shallow + blobless fetch)로 전송 병목 해소
- GitOps 기반 지속 배포 체계 정착 — 핵심 서비스 저장소별 주 100회 내외의 안정적인 배포 수행 및 신규 기능의 프로덕션 도달 시간 단축
- 다중 저장소 MasterData 파이프라인 구축 — 기획 데이터 변경 사항을 하위 서비스에 자동으로 일괄 전파하고, 파이프라인의 무한 반복 실행을 원천 차단

2-2. 공용 Toolchain Image 계층 구조 + 자동 버전 사이클 (2025.09 ~ 현재)

문제

GitLab CI 표준화 대상 repo마다 각 잡이 alpine 베이스에 helm·helmfile·kubectl·crane·aws-cli를 매번 설치해 빌드 환경 프로비저닝에 평균 5분이 걸렸고, 도구 버전이 consumer repo마다 흩어져 신버전 추가 시 모든 repo를 손대야 했습니다. `:latest` 의존으로 재현성과 취약점 추적도 어려웠습니다.

접근 전략

- **Layer 1 (Mirror)** - 외부 레지스트리(Docker Hub, GHCR)의 도구 이미지를 multi-arch manifest(여러 CPU 아키텍처를 묶은 이미지 목록)는 보존한 채 `crane copy` 로 사내 Harbor에 미러링
- **Layer 2 (Custom)** - Layer 1 위에 도구를 미리 설치해 둔 이미지(helmfile-tools, base-tools, node-build, rclone-backup, aws-cli-tools)를 빌드해 consumer repo가 image pin만으로 도구를 즉시 쓰도록 표준화
- **Tier 1~2 SSOT**(단일 관리 기준) - `.toolchain_build_custom` 공용 템플릿 + `TOOLCHAIN_*_TAG` 중앙 변수를 도입해 consumer repo가 image tag를 직접 추적하지 않도록 전환
- **공용 CI 템플릿 SSOT** - repo마다 복사해 쓰던 CI 잡을 공용 템플릿 repo로 추출해 `include` 로 참조. auth(정적 키 없는 AWS 인증) / build(kaniko) / deploy(ArgoCD) / backup(Google Drive) 잡 템플릿과 공용 앵커(git-ssh-rules·job-config·packages)를 한곳에서 관리해 복사본 간 설정 드리프트 제거
- **자동 버전 사이클** - 주간 스케줄이 (1)을 돌리고, 사람이 파이프라인을 실행하면 (2)~(4)가 자동 MR로 이어짐. 사람의 판단 지점은 파이프라인 실행과 Tier 2 MR 머지 두 곳:
 - (1) upstream 신버전 감지
 - (2) Layer 1 mirror auto-MR
 - (3) Layer 2 rebuild + Tier 2 TOOLCHAIN_*_TAG 동기화 auto-MR
 - (4) consumer repo(template provider repo 제외)에 자동 전파

트리블슈팅

- 자동 버전 상향 중 helmfile 1.5.10이 helm 3.18.6 이상을 요구하는데 helm은 3.16.4에 고정돼 있어 `build_custom` 빌드가 깨짐.
도구 간 호환 요구사항은 upstream 릴리스 노트로만 알 수 있어 자동 감지가 불가능하다고 판단, helm·helmfile·kubectl에 minor-guard를 도입해 현재 minor 안의 patch만 자동 상향하고 minor·major는 사람이 호환성을 확인한 뒤 올리도록 분리(`*_TARGET_MINOR` 환경변수로 계획된 minor bump 시 1회 override)

성과

- CI 빌드 환경 프로비저닝 시간 5분 → 약 10초 (약 95% 단축), 사내 repo 전반에 도구 버전 일관성 100% 확보, 버전 거버넌스 4-tier(ARG → 중앙 변수 → consumer 분리 → lint drift 감지)로 흩어져 있던 버전 지정 지점을 한 흐름으로 정리
- helm·helmfile·kubectl·crane·rclone 신버전 전파 자동화 — 파급이 큰 이미지 리빌드는 스케줄이 아닌 사람 검토 게이트 뒤에 배치
- 컴포넌트를 리스크 티어(T1 주간 / T2 격주 / T3 월간 + pre-flight)로 나눠 신버전 감지 → 자동 MR → Slack 알림 → wave 순차 적용 자동 업그레이드 파이프라인 구축(MAJOR_PIN 가드·T3 atomic 롤백 포함). docker buildx로 amd64 + arm64 multi-arch 이미지를 빌드, Layer 1 미러링으로 외부 레지스트리 의존 제거

프로젝트 3: 중앙 로깅·관측 플랫폼 (로그 수집 → 무결성·HA → 제품 분석)

기여도 100% (로그 수집·무결성·제품 분석까지 전 과정 단독 설계·구축) 2024.11 ~ 현재

로그가 분산된 온프레미스 환경에서 중앙 집중식 로깅 체계를 3단계로 고도화하고, 로그 수집 계층의 데이터 무결성과 고가용성(HA)을 강화한 뒤, 축적된 EFK 파이프라인을 게임 유저 리텐션 분석 플랫폼으로 확장했습니다.

3-1. 중앙 집중식 로깅 시스템 구축 (ELK → EFK → ECK Operator 3단계 고도화, 2024.11 ~ 현재)

문제

온프레미스 환경에서 서버 및 컨테이너 로그가 분산되어 있어 장애 발생 시 원인 파악에 많은 시간이 소요되었으며, 통합 모니터링 체계도 없었습니다.

접근 전략

- **ELK → EFK 전환** - 노드별 Filebeat 수집 + 중앙 Logstash 가공 구조에서 JVM 기반 Logstash 티어의 메모리 부담이 커, 수집은 경량 Fluent-bit DaemonSet, 파싱은 Fluentd로 교체
- **Operator 전환** - Elastic 공식 Helm 차트 2년 이상 정제 → ECK Operator의 CRD 선언적 관리로 이전

- **AWS(EKS) 클라우드 이전 및 자동화** - 온프레미스 EFK 스택을 AWS(EKS) 맞춤형으로 템플릿화해 배포하고, 로그 수명주기(ILM)와 S3 스냅샷 저장은 IRSA를 통해 정적 키 없이 자동화

트러블슈팅

- 온프레미스 Elastic Stack 신버전 자동 갱신 중, 릴리스 피드에는 등록되었으나 레지스트리에 실제 배포되지 않은 이미지를 참조해 `ImagePullBackOff` 발생. 즉각 롤백을 시도했으나 ECK Admission Webhook의 다운그레이드 거부 정책으로 복구 경로가 단절되고 Operator 장애로 확산.
배포 장애 자체보다 '롤백 불능 상태'를 핵심 운영 리스크로 정의하고, 업그레이드 스크립트에 이미지 레지스트리 실존 검증, 클러스터 헬스 사전 점검, 메이저 버전 상향 시 스냅샷 백업 경고, Webhook 제약을 고려한 안전 복구 절차를 체계화해 재발 방지

성과

- 분산 노드 및 휘발성(Ephemeral) 파드 로그의 중앙 집계 체계 구축 — 파드 재생성·종료 시 발생하는 로그 유실을 원천 차단하고, Kibana 단일 뷰 기반의 시계열 추적으로 장애 원인 규명 시간(MTTR) 대폭 단축
- ECK Operator 전환을 통한 관리 자동화 및 최신화 — TLS 인증서 갱신·계정 프로비저닝·노드 교체의 수동 부담을 제거(CRD 버전 선언만으로 롤링 업그레이드 보장)하고, 2년 이상 정체되었던 Elastic Stack 메이저 버전 업그레이드 완료 (8.x → 9.x)

3-2. Fluent-bit 로그 수집기 데이터 무결성 & HA 강화 (2026.03 ~ 현재)

SQLite 오프셋 DB를 노드 로컬 스토리지(`hostPath`)에 저장해 파드가 재시작되어도 읽은 위치가 유지되도록 구성(재색인·중복 0건 달성), 프로세스 다운 없이 로그 수집만 멈추는 비정상 정체(Stall) 상태를 감지하는 모니터링 알람을 연계해 미감지 로그 유실 방지

3-3. 게임 유저 리텐션 분석 플랫폼 (EFK 로그 → 제품 분석, 2026.06 ~ 현재)

장애·검색용 EFK 스택을 Elasticsearch Transform(5분)이 이벤트를 사용자당 1행으로 집계하는 제품 분석 플랫폼으로 확장 — Kibana 대시보드로 D+1~D+30 리텐션·코호트 제공, 별도 SaaS 없이 구현

프로젝트 4: 개발 생산성 도구 구축 (APK 배포 봇 · 문서 자동화 · LLM 서비스 · Git 미러링 · 정적 파일 서버)

기여도 100% (전체 시스템 설계 및 개발) 2025.02 ~ 현재

개발팀의 반복적인 수동 작업을 자동화하고 생산성을 높이기 위해, 5가지 사내 도구를 설계·개발·운영했습니다.

4-1. Slack APK 배포 자동화 봇 (4주, 2025.02)

Jenkins APK 빌드 완료 시 Slack으로 완료 메시지·다운로드 QR을 자동 전송하는 Python 봇을 Kubernetes에 배포 — QA 전달 시간 90% 단축, 버전 혼동 제거

4-2. GitLab-Google Drive 문서 자동화 시스템 (2주, 2025.06)

GitLab push를 트리거로 문서를 Google Drive에 rclone으로 단방향 동기화 — `--checksum` 비교로 실제 변경분만 전송, 바뀐 디렉터리만 부분 clone(sparse)해 대용량 저장소 전송량 절감, 문서 관리 시간 80% 단축(10분 → 2분), GitLab이 단일 기준이고 Drive 측 수정은 다음 동기화 때 덮어쓰는 구조

4-3. 사내 LLM 서비스 구축 (4주, 2025.11)

온프레미스 GPU 서버에 Ollama·Open WebUI를 docker compose로 올려 사내 LLM 직접 운영(개발팀 사용)

4-4. Git 저장소 미러링 도구 개발 및 Retry API 후속 강화 (4주 초기 개발, 2026.02 ~ 현재)

CodeCommit·GitLab·GitHub 간 수동 동기화 누락과 지연을 해소하기 위해 Go 기반 미러링 도구 git-bridge 자체 개발 및 오픈소스화 — 10개 저장소의 일 평균 30건 동기화를 100% 자동화하고, 실패 재시도 및 Slack 알람 연동으로 동기화 현황 실시간 추적

4-5. 정적 파일 서버 자체 개발 (오픈소스, 2026.03 ~ 현재)

유지보수가 중단되고 핵심 기능(UI·검색·ZIP 일괄 다운로드)이 없던 기존 파일 서버를 대체하기 위해 Go 기반 정적 파일 서버 자체 개발 및 오픈소스 공개 — 자체 Helm 차트와 NFS 스토리지를 연계해 클러스터에 배포하고 기존 도구를 완전 대체, 일 평균 30건의 다운로드와 주 5회 APK 배포 파이프라인을 안정적으로 처리

프로젝트 5: Kubernetes 클러스터 운영 고도화

기여도 100% (단독 설계 — 표준 절차를 자동화 스크립트로 코드화해 팀이 재사용) 2025.12 ~ 현재

안정적인 클러스터 운영을 위해 모니터링과 Kubernetes 업그레이드·Helm 관리를 자동화하고, 네트워크 컨트롤러를 Gateway API로 전환했으며, 스토리지 병목 해소와 배포/업그레이드 파이프라인 정비까지 이어갔습니다.

5-1. 모니터링 & 알림 고도화 (2025.12 ~ 현재)

kube-prometheus-stack 기반 커스텀 알림·Grafana 대시보드를 설계하고 DB·node-exporter·Cilium 메트릭을 통합 — 알림 규칙을 컴포넌트 그룹별로 묶고 심각도(severity)별로 나눠 보내며, 상위 알림 발생 시 하위 알림 억제(inhibit)로 같은 장애의 중복 통보 제거

5-2. Kubernetes 업그레이드 자동화 & Helm 차트 관리 프레임워크 (2025.12 ~ 현재)

Kubespray 업그레이드 절차와 업그레이드 후 9개 항목 검증(노드·etcd·API Server·인증서 만료 등)을 스크립트로 만들고 Helm 프레임워크·표준 템플릿 동기화로 버전 관리를 표준화 — 검증 시간 90% 단축, 인증서 갱신을 스크립트화해 만료 장애 제거

5-3. Ingress-nginx → NGINX Gateway Fabric(NGF) 마이그레이션 (2026.04)

온프레미스의 여러 ingress-nginx 인스턴스를 NGF 2.x 단일 컨트롤플레인과 클래스별 Gateway CR로 전환 — MetalLB IP 유지로 DNS·방화벽은 그대로 두고 사고 없이 cutover(클래스당 30~60초), wildcard 인증서로 수동 갱신 50% 감소

5-4. 스토리지 성능 최적화 — etcd / DB SSD 마이그레이션 + 빌드 I/O 격리 (2주, 2026.04 ~ 2026.05)

GitLab Runner(buildx)의 디스크 I/O 급증으로 HDD 기반 etcd 및 게임 DB에 초 단위 지연이 발생하는 병목을 진단해, Control Plane과 DB 워크를 SSD로 이전하고 빌드 워크로드를 별도 물리 서버 전용 VM으로 완전 격리 — 게임 DB COMMIT 지연 9~14초에서 ms 단위로 정상화, 빌드 중 인게임 접속 불가 및 서비스 지연 장애 0건 달성

5-5. Kubernetes 인프라 배포 자동화 및 멀티클러스터 GitOps(ArgoCD) 전환 (2026.03 ~ 현재)

문제 정의

Shell/awk 기반의 인프라 모노레포는 복잡한 YAML 조작과 환경별 분기 처리의 한계로 테스트 자동화가 불가능했고, CI 러너 중심의 Push 배포(`helmfile apply`)는 클러스터 상태 드리프트(Drift) 감지 불가 및 멀티클러스터 확장의 한계가 존재했습니다.

접근 전략

- **Python 표준 라이브러리 기반 툴링 리팩토링**: 서드파티 패키지 의존성을 배제한 Python3 모듈화 및 모듈별 1:1 단위 테스트(`unittest`) 체계를 도입해 로컬·CI 러너 간 실행 일관성 확보
- **단계별(Wave) 배포 및 업그레이드 자동화**: 컴포넌트 의존성에 따른 6단계 Wave 배포 체계를 수립하고, 차트 신버전 감지 시 Values diff 및 호환성 검증을 포함한 MR 자동 생성 파이프라인 구축
- **ArgoCD Pull-GitOps 컨트롤플레인 전환**: ApplicationSet(Matrix/Git Generator)으로 온프레미스·AWS 배포를 자동화하고, 기존 Helm 릴리스를 리소스 재생성 없이 ArgoCD 관리 상태로 무중단 이관(Adoption)
- **GitOps 거버넌스 및 안전장치 수립**: 최상위 계층 연쇄 삭제 방지(Cascading Prune 차단) 및 AppProject 허용 목록(Whitelist)을 적용해 장애 영향 범위(Blast Radius) 격리

주요 성과

- 온프레미스·AWS 멀티클러스터 플랫폼 릴리스를 단일 GitOps 컨트롤플레인에서 선언적으로 중앙 통제
- 레거시 Shell 스크립트를 단위 테스트가 보장된 Python 툴링으로 전면 마이그레이션해 배포 안정성 확보
- 차트 업그레이드 MR 자동화 및 Wave 배포를 통해 인프라 운영 오버헤드 대폭 감축
- 선언형 형상 자동 복구(Self-healing) 및 계층별 Prune 통제로 대규모 삭제 사고 위험 원천 차단

프로젝트 6: 인프라 보안 및 거버넌스 체계 구축

기여도 100% (설계, 배포, SSO 연동) 2026.04 ~ 현재

파편화되어 있던 계정 인증·시크릿 관리·원격 접근 경로를 표준화하여 사내 보안 거버넌스를 일원화했습니다.

6-1. 중앙 인증(SSO/OIDC) 체계 일원화 (Keycloak Operator, 2026.04 ~ 현재)

사내 도구(Harbor·ArgoCD·Vaultwarden)마다 개별 연동되어 파편화되었던 GitLab OIDC 인증 경로를 Keycloak Operator 기반 중앙 IdP로 통합. GitLab Group → Keycloak Group Mapper → ArgoCD Role로 이어지는 권한 흐름을 일원화해 인증 정책 중앙 통제 및 단일 로그인(SSO) 환경 구축

6-2. 시크릿 거버넌스 수립 — Vaultwarden · GitOps Secrets (2026.04 ~ 현재)

메일·메신저 등에 흩어져 있던 사내 공용 패스워드를 Kubernetes 기반 Vaultwarden으로 이관하고 SSO를 연동해 중앙 관리 체계 확립. 또한 Git에 평문 커밋되던 DB 접속 정보를 External Secrets Operator(AWS Secrets Manager 연동)로 전환하고, Harbor 클러스터 시크릿을 Sealed Secrets로 표준화하여 GitOps 환경의 자격증명 노출 취약점 원천 차단

6-3. 사내망 보안 원격 접속 환경 구축 (OpenVPN PKI, 2026.07)

동시 접속 50명 규모의 온프레미스 사내망 VPN 게이트웨이 구축. easy-rsa PKI 기반 개별 EC 인증서(prime256v1) 발급 및 tls-crypt 제어 채널 암호화(AES-256-GCM-TLS 1.2+)를 적용해 미인가 접속 시도를 핸드셰이크 단계에서 차단하고, 인증서 발급·폐기 전 과정을 먹통 셀 스크립트로 자동화

프로젝트 7: AWS EKS 프로덕션 게임 런칭 플랫폼 구축 (신규 외부 퍼블리싱 게임)

기여도 100% (단독 설계·구축·운영) 2026.06 ~ 현재 (구축·운영 중)

신규 외부 퍼블리싱 게임의 상용 런칭을 위해 eu-central-1에 그린필드 EKS 프로덕션 환경을 단독 구축하고(7-1), 분산 추적(7-2)과 Canary 무중단 배포(7-3)를 보강해 소프트 런칭(7-4)까지 운영했습니다.

7-1. 그린필드 EKS 프로덕션 인프라 구축 및 실패트랙 최적화 (eu-central-1, 2026.06 ~ 현재)

문제 정의

온프레미스 단일 클러스터 환경은 글로벌 트래픽 대응과 유연한 오토스케일링에 한계가 있어, 해외 상용 런칭을 위한 독립적인 고가용성 EKS 프로덕션 아키텍처 수립이 필요했습니다.

접근 전략

- **하이브리드 멀티클러스터 및 선언적 관리**: eu-central-1 리전에 그린필드 EKS 클러스터를 신설하고, 플랫폼 컴포넌트 전반을 GitOps(ArgoCD)로 선언적 일원화 관리
- **Karpenter 기반 컴퓨트 분리 최적화**: 게임 서비스는 On-Demand 노드로 안정성을 담보하고, CI 러너는 ARM 다중 패밀리를 Spot 풀로 다변화해 비용 절감 및 인스턴스 회수율 최소화
- **네트워킹·인증 자동화**: AWS Load Balancer Controller, ExternalDNS, Wildcard ACM을 연계해 Ingress 및 TLS 인증서 발급 전 과정을 자동화하고, 전체 ServiceAccount에 Pod Identity/IRSA를 적용해 정적 IAM 키 제거
- **로깅·시크릿 기반 구축**: eck-operator 기반 HA EFK 스택(3노드 3-AZ, EBS gp3, S3 스냅샷 SLM)과 External Secrets Operator를 도입해 신규 클러스터의 로그 수집·시크릿 참조 경로 확보
- **배포 경로 자격증명 격리**: 클라이언트 아티팩트는 Jenkins → S3 → CloudFront 파이프라인으로 전송하고, 서버 매니페스트는 S3 중계 및 SSM SendCommand를 통해 Bastion EFS에 원격 전달(SSH 키 상시 보관 배제)

트리블슈팅

- **Karpenter 노드 축소와 게임 세션 보호 양립**: 노드 회수 완전 차단(`karpenter.sh/do-not-disrupt`)은 유휴 비용을 유발하므로 배제하고, 세션이 Redis로 외부화된 Stateless 구조를 활용해 preStop drain + PDB(`minAvailable: 1`) + Disruption Budget(`nodes: 1`) 조합으로 1노드씩 안전한 순차 축소 달성
- **롤링 배포 후 유휴 노드 미회수 개선**: 메모리 Request 과다 설정(실측 101MiB 대비 2GiB)으로 발생한 노드 간 점유 불균형(92% vs 35%) 및 자동 축소 실패를 진단. Request를 512MiB로 현실화하고 Consolidation 조건을 조정해 빈패킹(Bin-packing) 밀도와 자동 노드 회수 정상화
- **신규 Pod 기동 중 트래픽 유입 차단**: Pod 기동 중 ALB가 트래픽을 선제 전달해 발생하던 간헐적 5xx 에러를 ALB Pod Readiness Gate 도입으로 해결. Target Group 헬스체크 통과 후에만 Pod를 Ready로 판정하도록 제어

주요 성과

- 해외 런칭을 위한 독립 AWS 하이브리드 멀티클러스터 구축 및 플랫폼 매니페스트 100% GitOps 선언적 관리 확립
- Bastion SSH 키를 비롯한 인프라 전반의 정적 자격증명 완전 제거
- 프로덕션과 심사용 워크로드를 단일 NodePool로 통합해 풀 분리 시 풀마다 기본 점유하던 최소 1노드의 비용 낭비 제거 및 bin-packing 밀도 극대화
- GitOps 매니페스트 및 CI/CD 전환 절차를 표준 런북으로 문서화하여 팀 내 인수인계 및 동일 환경 재구축이 가능하도록 함

7-2. OpenTelemetry 엔드투엔드 분산 추적 파이프라인 (OTel + Tempo, 2026.06 ~ 현재)

문제 정의

기존 관측 체계(EFK 로깅, Prometheus 메트릭)만으로는 마이크로서비스 간 호출 흐름과 지연 시간(Latency) 병목 구간, 분산 환경의 에러 전파 경로를 파악하는 데 한계가 있었습니다.

접근 전략

- **추적 스택 및 무침습 계측 환경 구축**: AWS EKS에 OpenTelemetry Collector와 Grafana Tempo(S3 백엔드, EKS Pod Identity로 정적 키 없이 연동)를 구축하고, OTel Operator Auto-instrumentation으로 애플리케이션 코드 수정 없이 언어별 SDK 주입
- **샘플링 지점 분리를 통한 관측 정확도·스토리지 비용 최적화**: RED 메트릭은 샘플링 전 100% Span에서 추출해 집계 정확도를 보장하고, 저장은 Tail Sampling(에러 전체 + 500ms 초과 요청 + 정상 요청 10%)으로 스토리지 비용 절감

주요 성과

- **3-Signal 상호 연계**: Metrics·Traces·Logs를 Grafana 단일 뷰에서 오갈 수 있게 연결해 서비스 간 호출 흐름과 병목 구간을 가시화(기존 로깅은 프로젝트 3, 메트릭은 5-1)
- **2계층 라우팅으로 트레이스 파편화 해결**: Collector 다중 replica에서 앱 Pod마다 gRPC 커넥션이 서로 다른 인스턴스에 고정되어, Pod 경계를 넘는 트레이스의 Span이 흩어지면 '에러 트레이스 전량 보존'과 Service Graph 조인이 깨질 수 있는 구조적 결함 식별
- 수신 계층(Layer 1)과 샘플링 계층(Layer 2)으로 분리하고 Headless Service + Trace ID 기반 라우팅(loadbalancing exporter)을 적용해, 한 트레이스의 Span이 항상 같은 샘플링 인스턴스에 모이도록 보장
- 실패트랙 검증: 커넥션 고정으로 replica별 시리즈 수가 1,650 : 541로 치우쳐 있던 분담이 Trace ID 해싱 후 초당 Span 4.09 : 5.18로 고르게 분산
- **샘플링 계층 알림 공백 보강**: 2계층 분리 후 기존 Collector 다운 알림이 샘플링 계층의 전면 장애를 감지하지 못하는 공백을 확인하고, 샘플링 계층 다운·버퍼 초과로 판정 전 버려지는 트레이스·버퍼 임계 근접·정책 평가 오류 알림 4종을 신설해 조용한 트레이스 유실을 감시

7-3. Argo Rollouts 기반 Canary 무중단 배포 전환 (Blue-Green → Canary, 2026.06 ~ 현재)

문제 정의

기존 Blue-Green 배포 방식은 승격(Promotion) 시 ALB Target Group을 교체하는 과정에서 일시적으로 Healthy Target이 0이 되는 공백이 생겨 간헐적 503 에러가 발생하는 한계가 있었습니다.

접근 전략

- **Canary 롤아웃 전환**: 동일 ALB Target Group 내에서 `maxUnavailable: 0` 및 add-before-remove(신규 Pod 기동 완료 후 구버전 회수) 전략을 적용해 구/신버전 일시 공존을 통한 503 레이스 컨디션 제거
- **Stateless 구조 전제**: 게임 서비스가 stateless HTTP(JWT + Redis/TypeORM으로 상태 외부화)라 전환 중 old/new에 트래픽이 섞여도 안전함을 확인. 다만 old/new가 동시에 떠 있으면 안 되는 변경(API 스펙 변경 등)은 이 전략 밖이라 정기점검 시간에 처리
- **배포 진입 및 종료 라이프사이클 동기화**: ALB Pod Readiness Gate를 통해 트래픽 수신 준비가 끝난 Pod만 트래픽 풀에 편입하고, 종료 시 Deregistration Delay(30초) → preStop(35초) → TerminationGracePeriod(60초) 순차 드레인 시퀀스를 확립해 In-flight 요청의 무결성 보장 (SIGTERM에서 스스로 in-flight 요청을 정리하는 .NET 서비스만 75초)
- **배포 전략 모듈화 및 컨트롤러 HA**: 공용 Helm 차트 내 배포 전략을 템플릿화해 프로덕션 게임 메인 서비스 1개에 Canary를 적용(공개 트래픽이 없는 같은 게임의 내부 서비스는 일반 Deployment 유지)하고, 컨트롤러 Leader Election HA 및 PDB를 구성해 노드 재배포 중에도 안정적인 롤아웃 지원

주요 성과

- 프로덕션 게임 배포 시 발생하던 ALB Healthy Target 0 공백 및 503 레이스를 원천 차단, 하위 호환 패치는 무중단 배포

7-4. 소프트 런칭 운영 (2026.07 ~ 현재)

성과

- 2026.07.14(KST) 소프트 런칭 개시 — 제한 공개로 연 소규모 실트래픽을 앞서 구축한 Canary 배포·3-signal 관측 위에서 안정적으로 받아냄
- 2026 Q4 글로벌 정식 런칭에 필요한 관측·배포·오토스케일링(HPA·Karpenter)을 모두 갖추(소프트 런칭 규모에서는 아직 스케일아웃이 걸리지 않음)
- 소프트 런칭 실트래픽 14일치 실측으로 워크로드 requests·limits를 재산정 — Pod당 관측 피크 기준으로 request를 정해 HPA 스케일아웃이 실제로 걸리도록 조정(과다 request는 사용량을 HPA 목표치 밑에 영구 고정해 스케일아웃을 한 번도 트리거하지 않음 — 기존 값에서 실제로 그랬음)

프로젝트 8: EKS Auto Mode 2차 클러스터 및 Site-to-Site VPN(WireGuard) 게이트웨이 구축

기여도 100% (단독 설계·구축) 2026.08 ~ 현재 (구축·전환 중)

문제 정의

신규 퍼블리싱 게임이 퍼블리셔 내부 인증 계정과 통신해야 했고, 이 구간은 공용 인터넷을 경유할 수 없어 VPC 피어링을 통한 사설 경로가 요구되었습니다. 그런데 기존 프로덕션 VPC의 대역이 피어링 조건과 맞지 않아 VPC와 그 위의 클러스터·데이터 계층까지 전면 재생성이 불가피했고, 재구축되는 클러스터는 퍼블리셔의 요청에 따라 EKS Auto Mode로 구성해야 했습니다.

기존 클러스터는 관리형 노드그룹 위에 Karpenter를 직접 운영하는 구조라 컴퓨트 영역의 Terraform·state 관리 포인트가 비대했고, 노드 수명주기·스팟 중단·핵심 애드온을 직접 관리하는 오버헤드도 컸습니다. 또한 VPC 피어링은 중계 라우팅을 지원하지 않아(non-transitive) 온프레미스에서 AWS 내부 ALB(private)로 가는 사설 경로는 별도로 확보해야 했습니다.

접근 전략

- **EKS Auto Mode 기반 컴퓨트 계층 관리형 전환**: 노드 프로비저닝, 수명주기 관리, Karpenter 부속 리소스(IAM·SQS·EventBridge) 및 내장 애드온(LB Controller·EBS CSI)을 AWS 완전 관리형으로 이관하고 선언적 NodePool만 유지하여 인프라 코드 복잡도 대폭 경감
- **IPAM 제약 대응 플랫폼 재설계(Re-platforming)**: 가용 서브넷 확장 불가 제약(/20 4개가 상위 /18을 정확히 소진)에 맞춰, 서브넷 물리 분리 대신 단일 private 서브넷 내 보안 그룹(Security Group) 기반 논리 격리 아키텍처로 전환(Aurora·Valkey·EFS 재배포)
- **동적 공인 IP(DDNS) 대응 Site-to-Site WireGuard VPN 구축**: 사무실 공인 IP가 유동이라 고정 IP가 필수인 AWS 관리형 VPN(Customer Gateway) 도입이 불가함에 따라, 온프레미스에서 AWS(EIP) 방향 아웃바운드 터널을 수립하고 peer endpoint를 의도적으로 비워 첫 핸드셰이크 패킷으로 변경된 IP를 자동 추적하는 게이트웨이 설계
- **플랫폼 컴포넌트 호환 마이그레이션**: 관측성·보안·CI 툴체인 등 16개 핵심 플랫폼 컴포넌트를 Auto Mode 사양에 맞게 재구성하고, 10개 컴포넌트를 ArgoCD Pull-GitOps 관리 체계로 전환

트러블슈팅

- **Auto Mode 시스템 테인트로 인한 부트스트랩 순환 교착 해소**: 기본 system NodePool의 `CriticalAddonsOnly` 테인트 때문에 자체 워크로드 노드가 프로비저닝되기 전까지 일반 Pod 용량이 0이라 ArgoCD Pod부터 Pending으로 멈추는 데드락 발생. NodePool·IngressClass 매니페스트를 GitOps 트랙에서 분리해 helmfile 기반 사전 부트스트랩 단계에서 선제 배포하도록 배포 순서 재정립
- **IngressClass 누락에 따른 로드밸런서 미생성(silent failure) 해결**: AWS Load Balancer Controller Helm 차트가 프로비저닝하던 IngressClass가, 컨트롤러를 내장한 Auto Mode에는 그 차트가 없어 만들어지지 않음. `ingressClassName: alb` 를 명시해도 매칭되는 컨트롤러가 없어 에러 로그도 없이 로드밸런서가 생성되지 않는 현상을 식별하고, Auto Mode 전용 IngressClass를 독립 컴포넌트로 정의해 클러스터 생성 직후 선행 배포
- **NodeClass API 그룹 변경 및 미지원 애드온 분리**: Auto Mode에서도 NodePool 작성은 그대로지만 NodeClass의 API 그룹이 `karpenter.k8s.aws/v1` EC2NodeClass에서 `eks.amazonaws.com/v1` NodeClass로 바뀌어 기존 매니페스트가 호환되지 않음. 스펙을 신규 CRD에 맞춰 재작성하고, Auto Mode가 내장하지 않는 EFS CSI Driver는 배포를 별도로 분리

주요 성과

- 컴퓨트 계층 Terraform 관리 포인트를 102개 → 17개로 83% 감축하고, 노드 수명주기·스팟 회수 오버헤드를 AWS 완전 관리형으로 이관

- 온프레미스·AWS 간 Site-to-Site VPN 게이트웨이로 내부 ALB를 공용 인터넷에 노출하지 않고 단일 내부 호스트명·단일 TLS 인증서 유지 — 대안인 환경별 public ALB 추가는 환경이 늘 때마다 이름·인증서·DNS 레코드가 두 배가 되어 기각
- 부분 장애(gray failure)를 차단하는 fail-fast 라우팅 — 정적 라우트를 Pod·노드 단위가 아닌 게이트웨이 라우터 단일 지점에 두어 경로가 사라지면 전체가 한 번에 실패하도록 구성(노드마다 두면 같은 서비스의 동작 여부가 Pod 배치에 따라 갈림)
- 터널이 Terraform 밖에서 의존하는 세 가지(라우터 정적 경로·게이트웨이 설정 파일·개인키)를 런북에 명시해 `terraform plan`에 보이지 않는 의존을 드러내고, 개인키는 여러 사람이 읽는 state 버킷에 평문으로 남지 않도록 SSM Parameter Store SecureString으로 완전 분리

너디스타

2023.03 ~ 2024.07

DevOps Engineer

게임·블록체인 기반 스타트업에서 DevOps 엔지니어로 근무하며 AWS에서 GCP로 대규모 클라우드 마이그레이션을 총괄했습니다.

비용 최적화와 글로벌 네트워크 성능 확보를 목표로, 게임 점검 시간을 활용해 최소 다운타임으로 전체 서비스를 이전했습니다. 소스 저장소는 온프레미스는 GitLab, 프로덕션은 GitHub로 이원화해 관리했습니다.

또한 MongoDB·CloudSQL·GA·Dune 등 멀티 소스 데이터를 BigQuery로 모아 Google Sheets까지 자동 갱신하는 데이터 수집 파이프라인을 구축해, 분석가의 일 2~3시간 수동 작업을 없앴습니다.

프로젝트 1: AWS → GCP 대규모 클라우드 마이그레이션 및 CI/CD 구축

기여도: 인프라 아키텍처 설계·마이그레이션 전략 수립 및 실행 총괄 70% (나머지 30%는 서버 개발팀의 애플리케이션 측 마이그레이션 협업), CI/CD 파이프라인 구축 100% 10개월 (2023.03 ~ 2023.12)

문제

AWS 비용이 지속 상승했고, 특히 상시 가동되는 게임 워크로드를 Fargate로 돌리며 발생하는 Pod 단위 과금이 큰 비중이었습니다. 비용 절감과 멀티 리전 게임 서비스를 위한 글로벌 로드밸런싱 확보를 목표로 GCP 마이그레이션을 추진했습니다.

접근 전략

- **3단계 순차 이전** - 1단계 개발 환경 GCP 구축·검증, 2단계 QA, 3단계 프로덕션
- **네트워크·인증** - Shared VPC(Host Project / Service Project 분리) 설계, GitHub Actions의 GCP 인증은 Workload Identity Federation으로 전환해 서비스 계정 키 발급·순환을 OIDC 단기 토큰으로 대체.
DNS는 서브도메인 위임(Hosting.kr → AWS Route53 → GCP Cloud DNS)으로 사전 검증하고, 최종 NS 변경만 점검 공지 후 cutover
- **컴퓨터 재배치** - AWS Fargate 워크로드를 GKE 표준 노드 풀로 이전. 상시 워크로드는 Pod 단위 과금이 VM 단위보다 불리해 동일 과금 구조인 GKE Autopilot 제외
- **스토리지 제약 우회·WAF** - Filestore SSD 최소 2.5TB 제약은 Compute Engine에 pd-balanced 1TB 디스크를 붙인 NFS 서버 직접 구축으로 우회, Cloud Armor로 리전 차단·IP 화이트리스트 WAF(웹 방화벽) 정책 구성
- **IaC·GitOps 파이프라인** - Terraform으로 GCP 인프라 IaC화(IAM 제외 100%), 이전 후 GitLab CI + ArgoCD GitOps CI/CD 파이프라인 구축, Helm Chart로 환경별 설정 표준화

트러블슈팅

- AWS ALB 기반 라우팅을 GCP 글로벌 로드밸런서로 전환하는 과정에서, 백엔드가 모두 unhealthy일 때 AWS ALB는 fail-open으로 트래픽을 통과시키지만 GCP LB는 이를 차단하는 차이 때문에 트래픽 라우팅 문제 발생.
GCP NEG(Network Endpoint Group) 구조를 분석하여 GKE 워크로드에 맞는 헬스체크·백엔드 설정으로 재구성
- GKE Ingress + BackendConfig의 healthcheck requestPath 응답이 누락되어 503 에러 발생. 해당 경로 응답을 보장하고 ManagedCertificate / FrontendConfig 설정과 서로 맞춰 외부 진입 트래픽 정상화
- Cloud CDN 마이그레이션 후 HTTP → HTTPS 리다이렉트 누락으로 게임 클라이언트 파일 다운로드 실패.
URL Map을 HTTP / HTTPS 두 개로 분리(`default_url_redirect.https_redirect=true`)하고 HTTP forwarding rule에 80 포트 target HTTP proxy를 별도 매핑하여 해결
- Cloud DNS 위임 후 Google Search Console에 도메인이 자동 등록되지 않아 검색 노출·소유권 확인 문제 발생. Search Console verification TXT 레코드를 Cloud DNS에 추가해 소유권 검증 후 재등록

성과

- 클라우드 비용 50% 절감 — AWS → GCP 마이그레이션 설계·수행, Fargate → GKE 표준 노드 풀 전환, MSP 리셀러 할인 확보
- 3단계 전략으로 전체 서비스 이전 완료 — 리소스 복사 방식으로 다운타임 최소화, 최종 DNS 전환은 2시간 내외 점검 공지 후 실행
- 전체 GCP 인프라를 Terraform으로 IaC화하고, 배포 이력을 Git 커밋으로 100% 추적 가능하게 함
- Workload Identity Federation 도입으로 GitHub Actions의 GCP 서비스 계정 키 관리 부담과 키 유출 리스크 제거

프로젝트 2: 게임 데이터 수집 자동화 시스템 개발

기여도 100% (Python 스크립트 개발 및 운영) 3개월 (2024.01 ~ 2024.03)

문제

게임·블록체인 데이터를 수동으로 수집하다 보니 분석가가 매일 2~3시간을 소비했고, 휴먼 에러로 인한 데이터 누락도 빈번했습니다.

접근 전략

- **멀티 소스 BigQuery 적재** - MongoDB는 Dataflow Flex Template ETL, CloudSQL은 BigQuery 직접 connection, Google Analytics · Dune은 각자 API(Dune은 별도 API Key)로 적재, 일관된 인덱스 스키마로 정규화
- **분석가용 Google Sheets 자동 갱신** - Python Cloud Function이 BigQuery 쿼리 결과를 Google Sheets API로 적재, Cloud Scheduler가 Daily(전일)·Monthly(전월) 두 주기로 자동 트리거
- **전체 인프라 Terraform IaC** - BigQuery Dataset · Cloud Function · Scheduler · Cloud Storage · Artifact Registry · IAM · Service Account를 코드로 관리

트리블슈팅

- CloudSQL → BigQuery 직접 connection(federated query)이 리전 불일치와 connection Service Account 권한 부족으로 실패. connection을 동일 리전에 생성하고 connection SA에 Cloud SQL Client 권한을 부여하여 해결
- Dune API의 execute → poll → fetch 비동기 실행 구조와 요청 한도로 결과 누락·타임아웃 발생. 실행 상태 폴링과 백오프로 결과 수신을 보장하고 API Key를 Secret Manager로 분리
- Daily 재실행 시 동일 행이 BigQuery에 재적재되는 문제 발생. 정기 실행되는 중복 제거 Cloud Function에 MERGE / ROW_NUMBER() 정합성 로직을 적용해 적재를 멍등화(재실행해도 결과 동일)하고 누락·중복 0건 유지
- BigQuery 결과를 Google Sheets API로 적재할 때 단일 시트 1천만 셀 한도·쓰기 쿼터와 분당 요청 한도(429)에 걸려 갱신 실패. values.batchUpdate 체크 분할과 시트 분할로 셀 한도를 우회하고, 호출 pacing(throttle)·배치 처리·지수 백오프로 쿼터 내에서 동작하도록 구현

성과

- 데이터 수집 자동화 95% 달성 (일 2~3시간 수동 작업 제거), 휴먼 에러로 인한 데이터 누락률 0%
- 전체 인프라 Terraform IaC화로 동일 파이프라인을 다른 GCP 프로젝트에 30분 내 재구축 가능, 변경 이력은 Git으로 100% 추적

프로젝트 3: AI 워크로드용 GPU 인프라 표준화 및 확장 (Windows → Linux · 세팅 자동화 · 사내 / GCP / 외부 IDC)

기여도 100% (인프라 설계·구축·운영) 6개월 (2024.02 ~ 2024.07)

문제

사내 GPU 서버가 Windows로 운영돼 이미지 생성 모델(Stable Diffusion 계열)을 돌릴 때마다 엔지니어가 실행 환경을 직접 세팅해야 했습니다. 드라이버·CUDA·프레임워크 조합을 손으로 맞추다 보니 같은 환경을 재현하기 어려웠고, 장비는 있는데 쓰기까지 시간이 걸렸습니다.

접근 전략

- **Windows → Linux 표준화** - 사내 GPU 서버를 Ubuntu 22.04로 전환, 세팅 스크립트가 드라이버 스택(nvidia-driver-535 · CUDA 11.5 · cuDNN 8.6)과 nvidia-docker 설치·검증까지 처리해 누가 실행하든 동일 환경 재현. 그 위에 이미지 생성 워크로드(Stable Diffusion WebUI · LoRA 학습용 kohya_ss · ComfyUI)를 표준 구성으로 배치
- **GCP 클라우드 확장** - NVIDIA A100 40GB 2장 VM(a2-highgpu-2g)을 Terraform IaC로 구축, 리전·존별 A100 가용성 확인과 GPU 쿼터 상향을 거쳐 앞선 마이그레이션의 Shared VPC 서브넷에 연결해 네트워크 표준 유지
- **외부 IDC 20대 증설·워크로드별 카드 분리** - 같은 세팅 스크립트로 GPU 서버 20대 구축. 학습은 모델·배치가 카드 하나에 들어가지 않아 VRAM과 다중 GPU 연결(NVLink)이 필요하고, 이미지 생성 추론은 요청마다 독립적이라 대수에 비례해 처리량 증가 → 학습·대용량 작업은 GCP A100, 대량 추론은 IDC의 소비자용 RTX 40 시리즈로 분리 배치
- **퍼블리셔 협업 고도화** - 신규 퍼블리싱 계약 체결로 수요가 이어져 퇴사 시점까지 퍼블리셔 측과 협업하며 GPU 환경 고도화

성과

- Windows 수동 세팅을 Linux + 자동화 스크립트로 대체 — 같은 스크립트로 외부 IDC 20대까지 구축해 1대용 절차가 GPU 플릿 전체의 표준 구축 절차가 됨
- GPU 서버를 손으로 구축하지 않고 Terraform 코드로 관리해 재구축·변경 추적을 표준화 — 기존 GCP IaC 저장소에 그대로 편입
- GPU 자원을 사내 서버 1대에서 사내 10대 · GCP A100 · 외부 IDC 20대 세 곳으로 확장 — 워크로드(다중 GPU 학습 vs 요청 단위 추론)에 맞춰 카드 등급 분리 배치

아이오차드

2022.04 ~ 2023.03

Infra Engineer

PaaS(Kubernetes + OpenStack + Ceph) 기반 인프라 솔루션을 고객사에 구축하고 기술 지원을 제공했습니다. 금융권·공공기관 프라이빗 클라우드 인프라를 설계·구축하고, 고객사 운영팀 교육을 직접 맡았습니다.

프로젝트 1: 고객사 맞춤형 PaaS 솔루션 구축

기여도 100% (인프라 설계 및 Kubernetes 클러스터 구축) 11개월 (2022.04 ~ 2023.03)

문제

고객사마다 요구하는 환경과 서버 스펙이 상이하여, 표준화된 솔루션으로는 대응이 어려웠습니다.

접근 전략

- **HA 클러스터 설계** - 고객사 요구사항에 맞춰 Kubernetes 클러스터를 HA(High Availability) 구성으로 설계하고 OpenStack으로 가상머신 관리 환경 구축
- **통합 스토리지** - Ceph로 블록·파일·오브젝트 스토리지 통합 제공

트러블슈팅

- **인증서 만료 주기 개선** - Kubernetes 클러스터 인증서의 1년 주기 만료로 인한 서비스 장애 리스크 및 매년 반복되는 갱신 작업(고객사당 연간 약 5시간) 발생. kubeadm 내장 갱신 명령(`certs renew`)의 특정 버전(v1.15~1.16) 미갱신 버그를 분석하여, 버전에 영향받지 않는 10년 유효기간의 자체 인증서 재생성 및 컴포넌트 재기동 스크립트(`update-kube-cert`)를 수립해 전 고객사에 무장애 적용

성과

- 5개 고객사에 HA 구성 PaaS 솔루션 구축·납품 후 안정 운영
- Ansible 기반 서버 프로비저닝·구성 관리 자동화로 고객사별 배포 시간 단축, 환경 불일치(Configuration Drift) 0건 달성
- 인증서 만료 주기를 1년에서 10년으로 연장하여 만료 장애 리스크 원천 차단 및 운영 중이던 전체 고객사 10곳 기준 연간 운영 공수 약 50시간 절감

프로젝트 2: DP사 Kubernetes 운영 교육 프로그램 설계·운영

기여도 100% (교육 설계 및 진행) 6개월 (2022.06 ~ 2022.11)

문제

DP사 운영팀이 Kubernetes·컨테이너 기반 인프라 운영 경험이 부족하여, PaaS 솔루션 도입 후 자체 운영에 어려움이 예상되었습니다.

접근 전략

- **단계별 커리큘럼** - Kubernetes 기초부터 클러스터 운영·트러블슈팅까지 단계별 교육 커리큘럼을 설계하고 실습 환경을 구성해 실무 중심으로 운영
- **PaaS 스택 병행 교육** - OpenStack·Ceph 스토리지 운영 교육을 병행해 전체 PaaS 스택 운영 역량 확보

성과

- 수강자 평균 10명 · 회당 교육 4시간 규모로 커리큘럼을 운영하여 DP사 운영팀의 Kubernetes 자체 운영 체계 확립
- 교육 완료 후 고객사 기술 문의 건수 감소, 고객사가 장애에 스스로 대응할 수 있게 됨
- 교육 자료를 표준화하여 이후 타 고객사 교육에도 재사용